

yt-factory — evidence of the upload flow

Project **yt-factory-510411** · OAuth client 147757322186-
hj1csktsf05hq0jj4g43frlhmis9r1sr.apps.googleusercontent.com · scope youtube.upload

This application has no user interface to screenshot: it runs unattended on a server and publishes to one channel belonging to its owner. The equivalent evidence is below — the log of a real upload, and the source code that performed it. The run can be inspected in full at github.com/beautyapp800-ctrl/yt-factory-daily/actions/runs/37185441013, and the source at github.com/beautyapp800-ctrl/yt-factory-daily.

1. Log of a real upload

GitHub Actions run 37185441013, 4 October 2026. The video was uploaded private with publishAt set three days ahead, and its thumbnail set in a second call. The video is youtu.be/gRePZohxtuM, on the owner's own channel.

```
2026-10-04 08:01:38,414 | INFO      | seo      | title (62 chars): 10 Stoic Lessons To Keep Personal
Promises When No One Watches
2026-10-04 08:01:38,414 | INFO      | seo      | 13 tags, 12 chapter lines, description 1111 chars
2026-10-04 08:01:39,399 | INFO      | thumbnail | background scene_025_01.jpg skipped: OCR reads 3
characters on it
2026-10-04 08:01:40,697 | INFO      | thumbnail | background scene_028_00.jpg (contrast 87.7, rim
53.6, left 97.4, detail 14.0, ranked 2 of 60, 1 skipped for lettering)
2026-10-04 08:01:40,697 | INFO      | thumbnail | type: 'WHEN NO' at 180px over 'ONE WATCHES' at
94px, block 611x266 = 37% of the frame height (1 of 13 word pairs were on the table, 1 combinations
tried)
2026-10-04 08:01:40,697 | INFO      | thumbnail | thumb_small.jpg readable: at 210x118 a capital in
the smaller line is 13px (need 9), contrast 20.1:1 white and 9.7:1 amber (need 4.5:1); OCR read 50%
of the words (informational)
2026-10-04 08:01:40,951 | INFO      | upload    | uploading final.mp4 (327 MB) as '10 Stoic Lessons
To Keep Personal Promises When No One Watches', privacy private, going public at 2026-10-
07T18:00:00Z
2026-10-04 08:01:42,295 | INFO      | upload    | uploaded 2%
2026-10-04 08:01:42,835 | INFO      | upload    | uploaded 10%
2026-10-04 08:01:43,625 | INFO      | upload    | uploaded 20%
2026-10-04 08:01:44,415 | INFO      | upload    | uploaded 30%
2026-10-04 08:01:45,230 | INFO      | upload    | uploaded 41%
2026-10-04 08:01:46,046 | INFO      | upload    | uploaded 51%
2026-10-04 08:01:46,764 | INFO      | upload    | uploaded 61%
2026-10-04 08:01:47,501 | INFO      | upload    | uploaded 71%
2026-10-04 08:01:48,309 | INFO      | upload    | uploaded 82%
2026-10-04 08:01:49,190 | INFO      | upload    | uploaded 92%
2026-10-04 08:01:52,843 | INFO      | upload    | uploaded: https://youtu.be/gRePZohxtuM in 11.9s (40
chunks, 0 retries)
2026-10-04 08:01:57,761 | INFO      | upload    | thumbnail set
2026-10-04 08:01:57,762 | INFO      | upload    | quota: requests that cost quota this run
{'videos.insert': 1, 'thumbnails.set': 1}. videos.insert: documented as 1 unit in the separate
uploads bucket (100 a day). thumbnails.set: documented as ~50 of the 10,000 general units a day. The
API does not report usage; the real figure is in Google Cloud Console > APIs & Services > YouTube
Data API v3 > Quotas & System limits.
```

2. The code that calls videos.insert with publishAt

The request body. When a video is scheduled it is uploaded **private** and YouTube is asked to publish it at a stated moment, which is what status.publishAt means.

pipeline/upload.py

```

104 def build_body(video, cfg, now=None):
105     """The videos.insert request body for this video."""
106     privacy = cfg.get("privacy_status", "private")
107     if privacy not in PRIVACY:
108         raise UploadError(f"privacy_status {privacy!r} in config.json must be one of
109     {PRIVACY}")
110     yt = cfg.get("youtube") or {}
111     tags = video.get("tags")
112     tags = json.loads(tags) if isinstance(tags, str) and tags else (tags or [])
113     status = {
114         "privacyStatus": privacy,
115         "selfDeclaredMadeForKids": MADE_FOR_KIDS,
116         "containsSyntheticMedia": CONTAINS_SYNTHETIC_MEDIA,
117     }
118     if cfg.get("schedule_publish", True):
119         # YouTube requires a scheduled video to go up private; it makes it public itself at
120         # publishAt. Asking for public AND a schedule is rejected, so privacy is forced here
121         # rather than left to trip the upload.
122         status["privacyStatus"] = "private"
123         status["publishAt"] = publish_at(cfg, now)
124     return {
125         ... (the request body returned here is sent to videos.insert)

```

The moment itself is computed through the real time zone, so 21:00 local is correct in both summer and winter:

pipeline/upload.py

```

81 def publish_at(cfg, now=None):
82     """When YouTube should make this video public, as RFC 3339 UTC.
83
84     config.publish_time local time, config.publish_days_ahead days from now. The queue of
85     finished videos lives on YouTube rather than on disk, because the runner is disposable
86     and seven 300 MB files cannot be carried between runs; scheduling three days out means
87     three days of failed runs change nothing a viewer sees.
88
89     The conversion is done through the real zone, not a fixed offset: Kyiv is UTC+3 in
90     summer and UTC+2 in winter, so 21:00 local is 18:00Z for part of the year and 19:00Z for
91     the rest, and an offset written down once would be wrong for half of it.
92     """
93     zone = ZoneInfo(cfg.get("timezone") or DEFAULT_PUBLISH_TIMEZONE)
94     hour, _, minute = (cfg.get("publish_time") or DEFAULT_PUBLISH_TIME).partition(":")
95     days = int(cfg.get("publish_days_ahead", DEFAULT_DAYS_AHEAD))
96     local = (now or datetime.now(timezone.utc)).astimezone(zone)
97     when = (local + timedelta(days=days)).replace(hour=int(hour), minute=int(minute or 0),
98                                                    second=0, microsecond=0)
99     if when <= local:

```

```

100         when += timedelta(days=1)
101         return when.astimezone(timezone.utc).isoformat(timespec="seconds").replace("+00:00", "Z")

```

And the call, a resumable upload of the rendered file:

pipeline/upload.py

```

266     request = service.videos().insert(part="snippet,status", body=body, media_body=media)
267     stats = {"bytes": size, "resumed_from": 0, "retries": 0, "chunks": 0,

```

3. The code that sets containsSyntheticMedia

The narration is synthetic speech and the artwork is generated, so every upload declares it. These are constants, not configuration: no setting can switch them off.

pipeline/upload.py

```

49 # Set on every upload, not read from config: see the module docstring.
50 MADE_FOR_KIDS = False
51 CONTAINS_SYNTHETIC_MEDIA = True

```

They are checked on the request itself immediately before it is sent, and the upload is refused rather than sent undeclared:

pipeline/upload.py

```

135 def assert_mandatory_flags(body):
136     """The two compliance flags, checked on the request itself just before it goes out."""
137     status = body.get("status") or {}
138     if status.get("selfDeclaredMadeForKids") is not False:
139         raise UploadError("status.selfDeclaredMadeForKids must be false on every upload")
140     if status.get("containsSyntheticMedia") is not True:
141         raise UploadError("status.containsSyntheticMedia must be true on every upload: "
142                           "YouTube requires AI-generated content to be declared")

```